AlgoViz: External SOAP Partner Integration

Contract-level integration of an external algorithm execution provider.

Team member	Roll number	Service responsibility
Anurag Tiwari	20251651028	Progress Service; Team Leader
Saurabh Singh	20251651089	Algorithm Service
Niraj Kumar Gupta	20251651066	Visualization Service
Mohd Rashid	20251651057	User Service

1. Context: partner edge and SOAP justification

AlgoViz integrates the fictional external provider **LabCompute Academic Execution Partner** through one operation, **bubbleSortTrace**, which returns a Bubble Sort execution trace for the Visualization Service. SOAP is chosen because this assumed legacy academic provider publishes a versioned WSDL/XSD contract for typed arrays, ordered steps, credentials and execution faults. AlgoViz keeps its student-facing services in REST/JSON and translates the partner messages inside a Spring Boot adapter. The example uses HTTPS and a contract-defined credential header; SOAP message-security extensions could be added by agreement, but WS-Security and distributed transactions are not implemented here.

Project-specific adaptation

The supplied brief is written for CampusEats and lists payment, delivery or SMS gateways. This submission explicitly adapts that partner category to AlgoViz: an external execution provider is relevant to the existing executeAlgorithm() contract, while the supplied Assignment 2 has no payment, delivery, SMS or OTP operation. This is a project-specific substitution of the listed gateway categories, not a claim that the brief lists an algorithm provider.

Boundary and scope

LabCompute is independently hosted outside AlgoViz; it is **not** the internal Algorithm Service renamed as a partner. AlgoViz still owns algorithm metadata and complexity information, visualization sessions, playback controls and student-facing errors. The selected Bubble Sort path delegates trace computation to the external partner; other algorithms retain their existing execution paths.

This is a design-time integration. The provider, endpoint, key and example identifiers are illustrative. No live gateway or server is required. Group ID: **17**.

2. Alignment with Assignment 2

The supplied service-contract PDF defines executeAlgorithm() in the Visualization Service. Its detailed central-operation specification (Task 4, p. 2) adds **Unsupported Data** and **Execution Failed** to the shorter Task 3 error list. This integration uses that detailed specification for the partner failure mapping.

Assignment 2 item	How this integration preserves it
Input: algorithm name	Bubble Sort selects bubbleSortTrace. Algorithm names remain part of the AlgoViz contract, not a new public SOAP API.
Input: input data	Adapter maps the array into BubbleSortRequest/values/item. This partner accepts 2-20 signed 32-bit integers.
Input: visualization speed	AlgoViz retains playback speed locally; it is not required for trace computation and is not sent to the partner.
Output: execution status	A valid partner response becomes Success; the selected partner fault becomes Failed / Execution Failed.
Output: visualization steps	stepNumber, action, leftIndex, rightIndex and state are mapped into AlgoViz visualization steps.
Output: current data state	sortedValues becomes the final current data state; each trace step supplies intermediate state.
Output: complexity	The response describes Bubble Sort: O(n^2) worst case, O(1) auxiliary space excluding trace storage.
Hidden implementation details	AlgoViz does not expose provider credentials, SOAP XML, worker details or vendor fault codes to students.

WSDL structure: all required components

Component	Definition in partner.wsdl
types	Credentials, BubbleSortRequest, BubbleSortResponse, ExecutionFault; reusable IntArray and TraceStep types.
message	CredentialsMessage, BubbleSortInput, BubbleSortOutput and ExecutionFaultMessage.
portType	AlgorithmPortType: bubbleSortTrace with input, output and ExecutionFault.
binding	AlgorithmSoapBinding: SOAP 1.1 over HTTP; document/literal; explicit credential header and SOAPAction.
service / port	LabComputeAlgorithmService / AlgorithmSoapPort with the HTTPS address.

The full WSDL is in partner.wsdl. This design adds a partner adapter, not a new user operation or a database table. Inter-service calls remain through contracts; the adapter does not read another service's database.

3. SOAP messages and HTTP binding

The three supplied XML files are authored examples of the request, success and alternative fault. Each uses requestId **av-run-0001** and the same partner namespace. Credentials are bound explicitly in the SOAP Header; the Body contains the element named by the corresponding WSDL message part.

```
Endpoint: https://api.labcompute.example/soap/v1/AlgorithmService
```

```
POST /soap/v1/AlgorithmService HTTP/1.1
Host: api.labcompute.example
Content-Type: text/xml; charset=utf-8
SOAPAction: "https://api.labcompute.example/algorithms/v1/bubbleSortTrace"
Content-Length: 648

<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope
 xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
 xmlns:alg="https://api.labcompute.example/algorithms/v1">
 <soapenv:Header>
  <alg:Credentials soapenv:mustUnderstand="1">
   <alg:clientId>algoviz-group-17</alg:clientId>
   <alg:apiKey>DEMO_ONLY_NOT_A_REAL_KEY</alg:apiKey>
  </alg:Credentials>
 </soapenv:Header>
 <soapenv:Body>
  <alg:BubbleSortRequest>
   <alg:requestId>av-run-0001</alg:requestId>
   <alg:values>
    <alg:item>3</alg:item>
    <alg:item>1</alg:item>
    <alg:item>2</alg:item>
   </alg:values>
  </alg:BubbleSortRequest>
 </soapenv:Body>
</soapenv:Envelope>
```

Content-Length is the UTF-8 byte count of soap-request.xml, including its final newline. SOAPAction is copied exactly from the binding; the Host and request path reconstruct the service/port endpoint. SOAP-over-HTTP uses the standard transport URI even when the endpoint is protected by TLS [1, 2].

Response and fault correspondence

soap-response.xml: BubbleSortResponse contains requestId, sortedValues, steps, timeComplexity and spaceComplexity, in that XSD order. The example sorts [3, 1, 2] into [1, 2, 3] with five compare/swap steps; indices start at zero.
soap-fault.xml: soapenv:Fault contains faultcode soapenv:Server, faultstring and detail/ExecutionFault. The detail contains requestId, code ENGINE_UNAVAILABLE and message, as declared in the WSDL.

Success uses HTTP 200 with text/xml; the SOAP 1.1 fault uses HTTP 500 with text/xml. Response and fault are alternative outcomes, not both returned for the same call [2].

4. Discovery and fault mapping

Modern discovery

At design time, AlgoViz searches the partner catalogue for the algorithm execution service and version 1, follows the WSDL URL, checks the operation, namespace, binding and address, and retains the reviewed partner.wsdl. At runtime the adapter uses a configured endpoint. No UDDI server or runtime discovery on every call is needed.

```
{
  "business": "LabCompute Academic Execution Partner (fictional)",
  "service": "LabComputeAlgorithmService",
  "endpoint":
    "https://api.labcompute.example/soap/v1/AlgorithmService",
  "wsdl":
    "https://api.labcompute.example/contracts/v1/AlgorithmService.wsdl",
  "tModelStylePointer": {
    "key": "urn:labcompute:algorithm:wsdl:v1",
    "overviewURL":
      "https://api.labcompute.example/contracts/v1/AlgorithmService.wsdl"
  }
}
```

The tModel-style pointer identifies the technical contract represented by partner.wsdl; it is a catalogue record, not a live UDDI registration.

5. Fault mapping into the existing contract

The adapter reads the SOAP fault even when the partner returns HTTP 500. The selected **ENGINE_UNAVAILABLE** fault maps to the existing **executeAlgorithm() - Execution Failed** case from Assignment 2 Task 4. AlgoViz returns the public execution status **Failed**; no partial trace is returned as successful.

```
Proposed REST serialization of the existing semantic error:
HTTP/1.1 503 Service Unavailable
Content-Type: application/json

{
  "executionStatus": "Failed",
  "error": "Execution Failed"
}
```

Assignment 2 specifies operation names and semantic errors, but no URL paths, JSON field names or HTTP status codes. The serialization and HTTP 503 above are explicit binding choices for this assignment, not falsely attributed to Assignment 2. The React interface can display "Algorithm execution failed. Please try again later." without exposing the vendor code or faultstring.

Before calling the partner, AlgoViz applies its existing errors: unknown algorithm - Algorithm Not Found; empty or malformed input - Invalid Input; unsupported data types or values beyond the selected partner limits - Unsupported Data. Unexpected provider failures or malformed traces become Execution Failed. Sanitized internal logs may retain requestId and a partner fault code, never the API key.

References and validation

Source contract: assgn2.zip / AlgoViz task-(1, 3, 4, 6).pdf, Task 3 and Task 4, pp. 1-2; service boundaries from Task_2 AlogViz Diagram.drawio.png; Saurabh and Niraj responsibilities updated as confirmed by the team. Project scope and Assignment 3 brief were also consulted.

[1] W3C, WSDL 1.1: https://www.w3.org/TR/wsdl.html
[2] W3C, SOAP 1.1: https://www.w3.org/TR/2000/NOTE-SOAP-20000508/